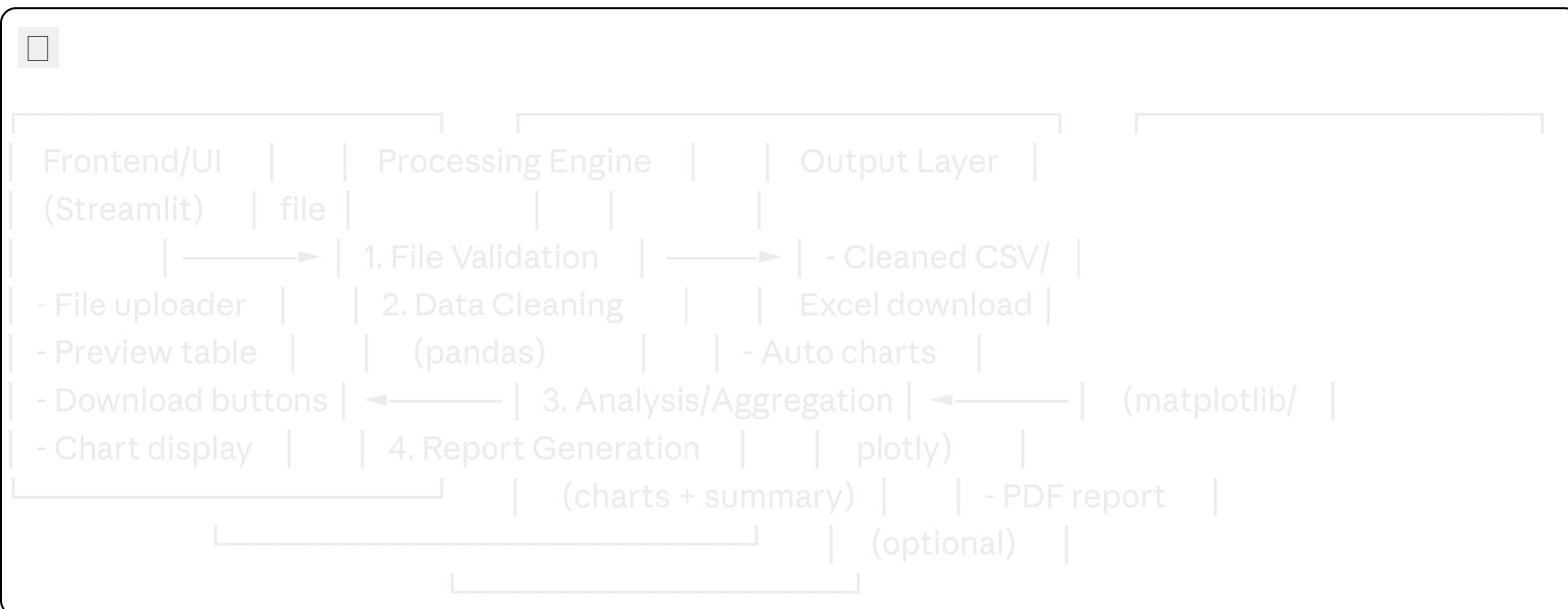


Project 2: Excel/CSV Report Automation Tool

Full Architecture + Build Roadmap

Positioning: "Automated Data Cleaning & Report Generator" — client uploads a messy CSV/Excel file (sales data, inventory, survey responses, etc.), tool cleans it and generates a polished report with charts automatically. Sell this as: "Turns your raw spreadsheet into a client-ready report in seconds."

1. Architecture Overview



Tech stack

Frontend + Backend combined: Streamlit (fastest way to ship this — no separate frontend/backend split needed, which means faster build + simpler deploy)

Data processing: Pandas (cleaning, aggregation), NumPy

Charting: Plotly (interactive, looks more "premium" than matplotlib for client demos) or Matplotlib for static exports

Report export: Pandas → Excel ([openpyxl](#)), optionally PDF via [reportlab](#) or [fpdf2](#) (adds a strong "wow" factor for clients — a formatted PDF report from raw data is a great sell)

Deployment: Streamlit Community Cloud (free, purpose-built for this, zero DevOps hassle)

Why Streamlit here instead of FastAPI+React (like Project 1): this project's value is in the *data processing logic*, not a custom UI experience. Streamlit lets you ship faster and it's genuinely well-suited for data tools — clients in this space actually expect a "dashboard" look, which Streamlit gives you out of the box.

2. Core Features (MVP scope)

File upload — accept CSV/XLSX, show a preview of raw data

Auto data cleaning:

Handle missing values (fill/drop, user-configurable)

Remove duplicate rows

Standardize column names (trim spaces, consistent casing)

Detect and fix common type issues (dates stored as text, numbers stored as strings)

Summary statistics — auto-generated: row/column counts, missing value report, basic stats (mean/median/min/max) for numeric columns

Auto-chart generation — based on column types detected:

Numeric columns → distribution/bar charts

Date columns → time-series trend chart

Categorical columns → count/pie charts

Export:

Download cleaned data as CSV/Excel

Download a summary report (Excel with multiple sheets, or PDF)

3. Step-by-Step Build Roadmap (4 days)

Day 1: Core Data Pipeline

- ☐ Set up Streamlit project structure (`app.py`, `utils/cleaning.py`, `utils/charts.py`, `utils/report.py`)
- ☐ Build file uploader (accept .csv, .xlsx) with pandas `read_csv`/`read_excel`
- ☐ Show raw data preview (first N rows) + basic info (shape, dtypes, missing value counts)
- ☐ Build the cleaning function: missing value handling, duplicate removal, column name standardization
- ☐ Add before/after comparison view (raw vs cleaned)

Day 2: Analysis + Chart Generation

- ☐ Auto-detect column types (numeric, categorical, date) using pandas dtypes + heuristics
- ☐ Build summary statistics generator (works for any uploaded dataset generically — this is the key engineering challenge: make it dataset-agnostic, not hardcoded to one sample file)
- ☐ Build auto-chart function using Plotly:
 - Loop through numeric columns → histogram/bar
 - Loop through date columns → line chart of trends
 - Loop through low-cardinality categorical columns → pie/bar chart
- ☐ Display charts in Streamlit using `st.plotly_chart()`

Day 3: Export + Report Generation

- ☐ Cleaned data export: `st.download_button()` for CSV and Excel formats
- ☐ Build Excel report export using `openpyxl`/`xlsxwriter`: multi-sheet workbook (raw summary, cleaned data, stats table)
- ☐ (Nice-to-have, strong differentiator) Build a PDF summary report using `fpdf2` — include key stats + embedded chart images
- ☐ Add error handling: corrupted files, empty files, unsupported formats, very large files (add a row-count warning)

Day 4: Polish + Deploy

- ☐ UI polish: sidebar for configuration options (e.g., "how to handle missing values" dropdown), clean layout with `st.columns()` for side-by-side views
- ☐ Add a sample dataset button ("Try with sample data") — critical for demos, lets prospects/clients test without needing their own file
- ☐ Write `README.md` with screenshots
- ☐ Deploy to Streamlit Community Cloud (connect GitHub repo, one-click deploy)
- ☐ Test live with a few different messy sample CSVs (different industries — sales, inventory, survey data) to confirm it's genuinely generic, not overfit to one file
- ☐ Record 30-60 sec demo video: upload messy file → show cleaning → show charts → download report

4. Deployment Checklist

Streamlit Community Cloud

Push code to GitHub (include `requirements.txt` with pinned versions: pandas, streamlit, plotly, openpyxl, fpdf2)

Go to share.streamlit.io → connect GitHub repo → select `app.py` as entry point

Deploy (usually live in 2-3 minutes)

Test the live URL with a few different files

Note: free tier apps sleep after inactivity too — same warm-up consideration as Render

5. What Makes This "Sellable"

"Try with sample data" button — lowers the barrier for a client to actually test it before hiring you; huge conversion boost on Fiverr

Generic, not hardcoded — make sure it genuinely works on *any* reasonably-structured CSV, not just your test file. This is what separates "a script" from "a tool" in a client's eyes

The PDF/Excel report is the real deliverable — most non-technical clients don't care about seeing raw data in a browser; they want something they can forward to their boss. Prioritize this if you're short on time.

Position it narrow when pitching: "Sales Report Automation for Small Retailers" sells better than "generic CSV tool" — same code, sharper pitch

Next possible steps

The core pandas cleaning + chart-generation code

Sample messy CSV dataset to test against (so it doesn't feel hardcoded)

Streamlit UI layout code